

DeepCausality

Dynamic Causality for a Dynamic World

For	Paul Hünermund, Jermain Kaminski, Beyers Louw
From	Marvin Hansen, Lead Maintainer of DeepCausality
Date	September 25, 2026
Purpose	Overview of the DeepCausality project

Summary

DeepCausality provides safe, dynamic causal reasoning for a world in which context changes continuously and reasoning must stay explainable and auditable. It is the Rust reference implementation of the Effect Propagating Process of dynamic causality.

1. Background

Computational causality and deep learning solve different problems. Deep learning excels at pattern matching, object detection, anomaly detection, and next-token prediction in large language models. Deep learning systems are non-deterministic: the same input may produce a different outcome. When production data diverge from the training data, hallucinations appear and reliability suffers. Other problems need explainable reasoning, deterministic decisions, and auditable end-to-end provenance. Computational causality provides that foundation.

Classical computational causality, from Pearl's structural causal models to Granger time-series analysis, assumes a fixed background spacetime and a static causal structure. That assumption traces back to Seneca's definition of causality two thousand years ago.

At the frontiers of science and engineering, that assumption breaks. Financial markets shift between regimes, autonomous vehicles process multi-scale feedback from sensors, and control systems must adapt to changing situations. In each case, reality changes continuously and defies static rules of causality.

2. Dynamic Causality

DeepCausality starts from a different premise: What if the rules of causality themselves change? In financial markets, the governing regime can shift. In space engineering, a spacecraft passes through subsonic, transonic, and supersonic regimes, and the physics differs in each. Classical causality cannot express these changes because it assumes causal invariance and a fixed spacetime.

DeepCausality rests on the Effect Propagating Process, which makes dynamics a first-class principle of causality. A single axiom generalizes causality as a spacetime-agnostic dependency. Detached from a fixed background spacetime, the process needs an explicit, dynamic context to reintroduce a well-defined one. That context supports modeling rich, dynamic, and complex operational environments. The process takes two core shapes: the causaloid for complex structure and the causal monad for strict sequencing. Rust lacks the higher-kinded types (HKTs) that monads require, so the DeepCausality project implemented them in the `haft` crate¹.

The causaloid is a self-referential isomorphic structure that uniformly represents a single causal relation, a collection of causes, or a hypergraph of causaloids. A collection may contain singletons, other collections, or hypergraphs, and a hypergraph node may contain a singleton, a collection, or another hypergraph. Causaloids can therefore represent arbitrarily complex structures. The causal monad entails no structure but guarantees composition. Causaloids and causal monads compose uniformly: effects propagate dynamically through rich causal structures and compose with each other.

Picture a ripple in a pond. Each ripple is an effect that propagates outward and produces the next ripple. DeepCausality lets you define how those ripples spread, what each one carries, and what happens when the rules of spreading themselves change. It supports four types of dynamic causal reasoning natively:

Type	Behavior
Static	Reasoning traverses a fixed causal graph along a predefined path.
Dynamic	Reasoning selects a path through a fixed graph at runtime. A subgraph or a shortest-path traversal answers a query.
Adaptive	The causal model dispatches to its next step by its own logic. A clinical model, for example, hands off to a “normal blood pressure” or “high blood pressure” subgraph depending on the latest vital reading, normalized to the patient.
Emergent	The graph is built and modified at runtime as the context changes.

Static, dynamic, and adaptive reasoning remain deterministic: their state space is bounded, their dispatch rules are known upfront, and formal verification stays practical. Emergent reasoning responds to a changing context, so it is non-deterministic and cannot be certified for safety-critical environments. DeepCausality also reasons across three modalities:

¹ https://crates.io/crates/deep_causality_haft

Modality	Applies when
Deterministic	The reasoning rules are certain.
Probabilistic	The reasoning contains uncertainty.
Mixed	Both modalities coexist in different parts of a causal model.

Multi-modal support enables modular models: engineers model, test, and certify deterministic parts in isolation, then combine them with modules for probabilistic reasoning or even an encapsulated LLM for classification. One DeepCausality code example combines deterministic reasoning with an LLM to classify incidents dynamically.

3. Examples

DeepCausality was designed from the ground up for engineering advanced dynamic systems. Its code examples simplify complex system designs by reformulating them as dynamic causal processes. They include simplified versions of:

- TCAS: Traffic Alert and Collision Avoidance System
- GPS blackout handling
- Flight envelope monitoring system
- Digital twin simulating weather impact on navigation

Each code example writes its entire decision process to an immutable log for external auditing. Beyond avionics, the examples model dynamic causal processes in physics, medicine, materials science, fluid dynamics, and quantum computing. Counterfactual fluid dynamics, which models alternate flows in response to changing conditions, has proven an interesting application of dynamic causality.

All code examples are available on GitHub:

https://github.com/deepcausality-rs/deep_causality/blob/main/examples/README.md

4. Capabilities

DeepCausality's single causal stack covers the full value chain of dynamic reasoning:

Capability	What it provides
Causal Discovery Language (CDL)	A domain-specific language that simplifies causal discovery. Identify causes when you need to.
Contextual dynamic causality	Encode process steps as composable causal logic, with or without an explicit context, to derive a causal verdict.
Causal state machines	Turn a causal verdict into an action on other systems or the outside world.
Effect Ethos	Deterministically verify which actions are permitted, and record each verdict in an immutable log that traces provenance end to end: from evidence to verdict to the permitted or denied action.
Model serving	Run the causal model synchronously or asynchronously in the framework of your choice. Native multi-threaded causal inference serves models at scale.

5. Context

DeepCausality supports three context configurations natively:

1. No context
2. Single context
3. Multiple contexts

The no-context case covers the corner case of decisions made relative to constants or to input values alone. A model gains a single context by referencing a context hypergraph, which multiple modules can share to stay consistent. Since 2023², DeepCausality has also supported reasoning over multiple contexts, which can be updated when a model needs a mutable context. This solves a subtle problem in modeling complex systems: reference data must remain immutable, yet several reasoning modules share intermediate artifacts. How do you update those artifacts so that other reasoning steps receive the update? An optional mutable context, shared with the modules that rely on those artifacts, solves this problem in a principled way. The update can come from inside the reasoning engine, from outside (for example, a data-feed update before the engine runs), or both (for example, when reasoning over multiple data feeds requires those artifacts at several stages).

² DeepCausality v.0.6 supports multiple contexts
<https://www.deepcausality.com/blog/announcement-multiple-contexts/>

DeepCausality's native context distinguishes four primitives:

- `Data<T>`
- `Space<T>`
- `Time<T>`
- `SpaceTime<T>`

The generic type `T` holds the embedded data, which may be a primitive type (i32, u64, etc.) or a complex struct. Users customize these four default primitives by reimplementing the corresponding interfaces. DeepCausality 0.8 added non-Euclidean³ context types for complex physics simulations across multiple regimes and non-linear time representations. Since 2025⁴, the UltraGraph data structure has backed both the context and the causal hypergraph. UltraGraph applies best practices from supercomputing to answer queries on graphs with a hundred million nodes in under a second on commodity hardware. Its dual-state architecture scales graph analytics to 10 billion nodes on a single server, which significantly reduces total cost of ownership.

6. Higher-Order Uncertainty

DeepCausality reasons over uncertainty with its *Uncertain<T>* type, which pairs a value with its inherent uncertainty, modeled as a probability distribution. *Uncertain<T>* offers these key capabilities:

- **Rich Distribution Support:** Create uncertain values from Point (precise), Normal (Gaussian noise), Uniform (range-bound), and Bernoulli (probabilistic Boolean) distributions.
- **Intuitive Operator Overloading:** Perform standard arithmetic (+, -, *, /), comparison (>, <, ==), and logical (&, |, !) operations directly on Uncertain types.
- **Lazy, Efficient Evaluation:** The engine evaluates the underlying computation graph lazily with intelligent sampling and statistical hypothesis tests such as SPRT, drawing only as many samples as a robust decision requires.
- **Comprehensive Statistical Analysis:** Compute statistics with methods such as `expected_value()`, `standard_deviation()`, and `estimate_probability()`.
- **Robust Decision-Making:** Decide under uncertainty with `to_bool()`, `probability_exceeds()`, and `implicit_conditional()`.

Uncertain<T> models the uncertainty of a value but assumes the value is present. Real-world data poses a more fundamental challenge: uncertainty about whether a value is present at all. A sensor fails intermittently; a patient misses a clinical measurement. Is 0.0 the true measurement, or is the data absent? Conventional primitive types cannot capture the probabilistic presence of a value.

³ DeepCausality v.0.8 adds support for non-Euclidean context
<https://www.deepcausality.com/blog/announcement-non-euclidean/>

⁴ UltraGraph 0.8: 1,300× Faster Graph Analytics
<https://www.deepcausality.com/blog/announcement-ultragraph-0-8/>

MaybeUncertain<T>⁵ addresses this higher-order uncertainty. It models values that are probabilistically present; when present, the value itself remains uncertain. Think of *MaybeUncertain*<T> as a “Probabilistic Maybe” type, hence its name. Instead of forcing a choice between a definite value and a definite None, it models the probability of presence itself, which matters for sparse or intermittently available data. It keeps uncertainly present data from propagating through a model as if it were certainly present, which could lead to incorrect conclusions. *MaybeUncertain*<T> lets causal pathways adapt to the quality and completeness of the available data and makes explicit what “available” means for the evidence behind each decision. DeepCausality treats both *Uncertain*<T> and *MaybeUncertain*<T> as first-class reasoning modalities.

7. Counterfactuals

DeepCausality handles counterfactuals⁶ natively across three channels:

- 1) Alternate Value: What if the value itself were different?
- 2) Alternate Context: What if the same reasoning occurred in a different context?
- 3) Alternate State: What if the state threaded through the reasoning were different?

The log channel cannot be altered, which keeps the reasoning log immutable and prevents evidence tampering. The three counterfactual channels are isolated, and each can be altered or preserved independently. DeepCausality marks each alteration in the log so that a later audit can recognize where a counterfactual scenario began.

⁵ DeepCausality Introduces the MaybeUncertain Type

<https://www.deepcausality.com/blog/announcement-maybe-uncertain/>

⁶ Counterfactuals via the Causal Monad

<https://www.deepcausality.com/blog/counterfactuals-via-the-causal-monad/>

8. Case Study

The Service Radar Project, an open-source network management solution, has adopted DeepCausality. Service Radar provides monitoring, security management, and real-time analytics for enterprise networks at scale. Its real-time anomaly detection engine, built on DeepCausality, models anomaly detection as a dynamic causal process. In a synthetic benchmark, the engine remained responsive at over 1 million events per second. United Airlines runs Service Radar in production, and Siemens is evaluating it.

9. Contact

DeepCausality is a project of the LF AI & Data Foundation, part of the Linux Foundation, and is developed by the Center for Dynamic Causality. For all inquiries, please contact:

Marvin Hansen,
Director, Center for Dynamic Causality
marvin.hansen@causalcenter.com

Project website: <https://www.deepcausality.com>

Center for Dynamic Causality: <https://www.causalcenter.com>

Linux Foundation: <https://lfaiadata.foundation/projects/deepcausality/>